Machine Learning - Assignment 2

Should an AI agent call a tool, or refuse?

AI agents often receive a user request and a list of tools they are allowed to use. Examples of tools are weather APIs, search APIs, calendar APIs, math functions, finance APIs, and database APIs.

The agent has to make a basic decision:

- use a tool, if at least one tool is relevant,
- refuse to use a tool, if none of the tools are relevant.

In this assignment, you will build machine-learning models for this decision.

The target column is **`can_answer`** :

Value	Meaning
`1`	At least one available tool is relevant to the user request.
`0`	No available tool is relevant. The agent should refuse to call a tool.

This is a binary classification task.

Why This Problem Matters

Calling a tool is not always harmless. A wrong tool call can:

- return irrelevant information,
- waste time, compute, or API money,
- send private user text to a tool that cannot help,
- perform an unwanted action, such as sending a message or changing a calendar event,
- make the agent sound confident when it should admit that its tools are not suitable.

There are two types of prediction mistakes:

Mistake	What happened?	Why is it bad?
False positive	The model predicts `can_answer = 1`, but the true label is `0`.	The agent may call a tool when it should refuse.
False negative	The model predicts `can_answer = 0`, but the true label is `1`.	The agent refuses even though it could have helped.

False positives are especially important in this assignment because they represent unwanted tool calls.

Dataset

Use this file:

`agent_tool_tasks.csv`

The file has **3,491 rows**. Each row is one agent task:

- the user query,
- the tools available to the agent,
- features describing the query, the tools, and the match between them,
- the target label `can_answer`.

The dataset is a simplified CSV extracted from JSON files in the Berkeley Function Calling Leaderboard (BFCL), published by the Gorilla project at UC Berkeley.

Original source:

- BFCL data folder: https://github.com/ShishirPatil/gorilla/tree/main/berkeley-function-call-leaderboard/bfcl_eval/data
- BFCL leaderboard: https://gorilla.cs.berkeley.edu/leaderboard

You do **not** need to download or parse the original JSON files. Use only the CSV provided with this assignment.

For a full explanation of every column, see **`DATA_DICTIONARY.md`**.

Important Dataset Notes

Pay attention to these important notes about the dataset:

- `can_answer` comes from benchmark construction. It is not a new manual annotation made for this course.
- `task_domain` and `task_complexity` are derived columns. They are useful, but they are not perfect ground truth.
- Aspect columns such as `query_aspects`, `tool_aspects`, and `aspect_coverage_ratio` are created using keyword rules. They may miss synonyms or misunderstand ambiguous words.
- Some columns are correlated. This is normal, but you should think about it during feature selection.
- A few rows have missing `tool_names` or `avg_param_description_length`. You must handle missing values.

Requirements

Section A - Data Exploration and Visualization (15 pts)

Explore the dataset before modeling.

You must include:

- at least **5 visualizations**,
- at least **3 different plot types**,
- one plot comparing `can_answer = 0` and `can_answer = 1`,
- one plot comparing live and non-live tasks using `is_live_benchmark`,
- one plot or table about `task_domain`,
- one plot or table about `task_complexity`,
- one plot or table about query/tool matching, for example:
  - `aspect_coverage_ratio`,
  - `aspect_overlap_count`,
  - `query_tool_token_jaccard`.

For every plot or table, write 2-4 sentences explaining the main observation.

Useful questions to ask:

- How imbalanced is the target?
- Are answerable tasks longer than refusal tasks?
- Are live tasks different from non-live tasks?
- Which domains are common?
- Do query/tool matching features differ between the two classes?

- Are tool-heavy tasks more or less likely to be answerable?

Light hint: some numeric columns have long tails or unusual outliers. For visualizations, it is okay to use log scales, clipping, or zoomed-in plots if you clearly explain what you did.

Section B - Preprocessing and Feature Engineering (25 pts)

Prepare the data for the supervised models.

B.1 Required Engineered Features

Create these 5 features:

1. **`required_params_ratio`**
   Formula: `total_required_params / total_params`
   If `total_params == 0`, set the value to `0`.
2. **`avg_params_per_tool`**
   Formula: `total_params / num_available_tools`
   If `num_available_tools == 0`, set the value to `0`.
3. **`query_avg_word_length`**
   Average length of the words in the raw `query`.
   Compute this from the actual words in the query.
4. **`query_mentions_number`**
   `1` if the raw `query` contains at least one digit, otherwise `0`.
5. **`tool_name_diversity`**
   Number of unique tool-name prefixes.
   The prefix is everything before the first `.`.
   Example: `math.factorial` has prefix `math`, `get_weather` has prefix `get_weather`.

B.2 Your Own Features

Create at least **5 more features**.

Examples:

- number of quoted strings in the query,
- `log(1 + total_params)`,

You can use these 2 features if you want, but only in addition to the 5 features you create yourself.

For each feature, briefly explain:

- what it measures,
- why it might help the model.

B.3 Cleaning and Transformation

You must do all of the following:

- handle missing values, including `avg_param_description_length`,
- apply at least one transformation, such as scaling or one-hot encoding,
- perform at least one feature exclusion or feature selection step.

The required removal of forbidden columns such as `task_uid`, raw `query`, raw `tool_names`, and `can_answer` does **not** count as your feature exclusion or feature selection step. Examples of valid steps include dropping highly correlated features, removing near-constant features, selecting features based on training-set correlation, or using model-based feature importance.

Do **not** use these columns directly as model inputs:

- `task_uid`,
- raw `query`,
- raw `tool_names`,
- `can_answer`.

You may use `query` and `tool_names` to create new features.

Categorical columns, such as `task_domain`, `task_complexity`, `query_aspects`, and `tool_aspects`, must be encoded if you use them in a model.

Columns such as `query_aspects`, `tool_aspects`, and `tool_names` may contain several values separated by a pipe (`|`). For example, `math|data_code` means that both aspects appear. For pipe-separated columns, you may encode the full string, split it into separate indicator features, simplify it, or exclude it, as long as you explain your choice.

Important rule: split the data before fitting imputers, scalers, encoders, feature selectors, or resampling methods. For example, if you impute with a median, calculate the median from the training set only. If you use one-hot encoding, make sure validation and test rows are transformed using the categories learned from the training set.

Section C - Classification: Predict `can_answer` (35 pts)

Train models that predict whether the agent can answer with its available tools.

C.1 Train / Validation / Test Split

Use:

- 80% train,
- 10% validation,
- 10% test.

The split must be stratified by `can_answer`.

Use `random_state = 42`.

Use:

- the train set to fit models,
- the validation set to choose hyperparameters,
- the test set only once for final results.

C.2 Models

Train these models:

1. k-Nearest Neighbors (you can use both ANN algorithms like LSH and KD-trees or the exact algorithm),
2. AdaBoost,
3. one more supervised model covered in class.

Also include a simple baseline, such as always predicting the majority class.

C.3 Hyperparameter Tuning

For each of the 3 main models:

- tune at least **2 hyperparameters**,
- choose the best setting using the validation set.

C.4 Evaluation

On the test set, report:

- confusion matrix,
- accuracy,
- precision and recall,
- F1-score.

Choose one main evaluation metric and explain why you chose it.

If the target is imbalanced, accuracy alone is usually not enough. You can handle the imbalance by choosing suitable evaluation metrics. Oversampling or undersampling are **not required**. If you use them, apply them only to the training set and explain why.

In your confusion matrix discussion:

- identify the false positives,
- identify the false negatives,
- explain what each mistake means for a tool-using agent.

Also include:

- one comparison plot of the model results,
- a short discussion of which model worked best,
- a short discussion of whether you prefer a conservative model that refuses more often or an aggressive model that calls tools more often. Explain your preference by analyzing the trade-off between false positive and false negative rates.

Section D - Clustering Agent-Tool Situations (15 pts)

In this section, do **not** predict `can_answer`.

Instead, use clustering to find common types of agent-tool situations. A "situation" means the combination of the query, the available tools, and the match between them.

The clusters do not need to match the target classes. For example, two tasks can both have `can_answer = 1`, but one may be a simple one-tool task and the other may be a complex task with many available tools.

D.1 Features for Clustering

Build a clustering feature matrix using features from at least **3** of the groups below. You do not have to use every column in a group.

Feature group	Possible columns
Query/tool matching	`aspect_coverage_ratio`, `aspect_overlap_count`, `aspect_mismatch_count`, `query_tool_token_jaccard`, `query_tool_action_overlap`
Query complexity	`query_word_count`, `query_multi_intent_score`, `query_specificity_score`, `query_unique_token_ratio`
Tool complexity	`num_available_tools`, `total_params`, `total_required_params`, `schema_rigidity_score`, `param_type_diversity`
Risk signals	`risky_tool_action_count`, `query_sensitive_data_signal`, `query_code_signal`, `query_temporal_signal`

Do **not** use:

- `can_answer`,
- `task_uid`,
- raw `query`,
- raw `tool_names`.

Scale the clustering features if the algorithm depends on distances.

D.2 Algorithms and Choosing Clusters

Apply exactly these two sklearn-friendly clustering approaches:

1. **K-Means**, using `sklearn.cluster.KMeans`.
2. **Agglomerative hierarchical clustering**, using `sklearn.cluster.AgglomerativeClustering`.

For K-Means:

- try several values for the number of clusters,
- use WCSS elbow plots and silhouette scores,
- choose a final number of clusters and justify it,

For agglomerative hierarchical clustering, compare linkage and distance choices:

- compare at least **3 linkage functions** from `single`, `complete`, `average`, and `ward`,
- compare at least **2 distance metrics** from `euclidean` /L2, `manhattan` /L1, and `chebyshev` /L-infinity,
- remember that Ward linkage only works with Euclidean distance.

For the hierarchical clustering experiments:

- try several values for the number of clusters,
- use silhouette scores and a dendrogram or linkage-distance plot,
- choose a final number of clusters and justify it.

When comparing hierarchical settings, explain how the linkage function and distance metric change the clusters you obtain.

It is okay if the best silhouette score is not very high. The goal is to find useful groups and explain them clearly.

D.3 Cluster Interpretation

For your final clustering:

- show cluster sizes,
- show mean or median feature values per cluster,
- compare clusters to `can_answer` only **after** clustering,
- compare clusters to `task_domain` or `task_complexity` ,
- identify if you can explain the clusters using the prediction from your best supervised model, for example:
  - are some clusters mostly false positives?
  - are some clusters mostly false negatives?
  - are some clusters mostly true positives or true negatives?
- give each cluster a short descriptive name.

Create a cluster profile table like this:

cluster	size	answer rate	key pattern	descriptive name

Here, "answer rate" means the percentage of rows in the cluster where `can_answer = 1` . Use it only for interpretation after the clustering is finished. The descriptive names do not have to be perfect. They should be based on the feature values in the cluster profile.

Presentation and Final Summary (10 pts)

Create a short presentation of no more than **6 slides**.
3-4 presentations will be chosen to be presented infront of the class. The goal is to learn from each other and share insights.

On the first slide of your presentation, include the name of the teaching assistant (TA) you are registered with, so we can group presentations by TA for the class presentation.

Required slides:

1. Problem and data.
2. Most important EDA insight.
3. Best classification model and test results.
4. Most important error analysis.
5. Clustering insight.
6. Final recommendation or limitation.

Also include a final summary table in your notebook with:

- best model,
- main metric,
- test performance,
- most important practical error,
- one dataset limitation,
- one suggested improvement.

Submit the presentation as a PDF.

Bonus Options

You may earn up to **20 bonus points** total.

Bonus 1 - L1, L2, and L-Infinity Clustering Comparison (up to 5 pts)

Compare whether the clustering result changes when the distance metric changes.

Use the same clustering feature matrix from Section D, or a clearly explained subset of it. Then run a clustering method that supports different distance metrics, such as `sklearn_extra.cluster.KMedoids` (use `pip install scikit-learn-extra` to get it).

Compare all three metrics:

- L2 / Euclidean distance, for example `metric="euclidean"` ,
- L1 / Manhattan distance, for example `metric="manhattan"` ,

- L-infinity / Chebyshev distance, for example `metric="chebyshev"`.

For each metric:

- try several values for the number of clusters,
- report a clustering-quality score, such as silhouette score,
- show cluster sizes,
- create at least one plot or table that helps compare the metrics.

Report:

- which metric gave the best result by your chosen score,
- whether the best metric also gave the most interpretable clusters,
- whether the cluster profiles changed meaningfully across L1, L2, and L-infinity,
- one short explanation of why the distance metric might matter for this dataset.

If you use KMedoids, clearly state that it uses **medoids**, which are actual data rows, instead of K-Means-style mean centroids.

Bonus 2 - Tool-Level Analysis (up to 15 pts)

Build a new dataset where each row is one unique tool name from `tool_names`.

Create at least 5 tool-level features, such as:

- number of tasks where the tool appears,
- average number of co-available tools,
- fraction of tasks with `can_answer = 1`,
- average query length for tasks containing the tool,
- tool-name prefix,
- average parameter counts of tasks containing the tool.

Then formulate one ML question about tools and answer it using a method covered in class.

Example questions:

- Can we cluster tools into common vs rare tools?
- Can we predict whether a tool usually appears in answerable tasks?
- Which tools are most associated with refusal cases?

Grading Summary

Section	Points
A. EDA and visualization	15
B. Preprocessing and feature engineering	25
C. Classification	35
D. Clustering	15
Presentation and final summary	10
Total	**100**
Bonus 1. L1/L2/L-infinity clustering comparison	+5
Bonus 2. Tool-level analysis	+15

Guidelines

Please read this section carefully before submitting the assignment.

Coding Guidelines

Your code should:

- run from top to bottom without errors or warnings,
- use clear section headers that match the assignment sections,
- use meaningful variable names,
- avoid reserved words as variable names,
- use constants where appropriate instead of repeating hard-coded values,
- include documentation or comments where they help explain non-obvious choices,
- avoid using the test set during model selection,
- explain any library used beyond the standard course libraries.

Use familiar packages when possible. If you install or use an additional library, briefly specify what it is used for in your notebook.

Submission Guidelines

Submit:

1. Notebook: `ML_HW2_ID1_ID2.ipynb`
2. HTML export: `ML_HW2_ID1_ID2.html`
3. Presentation PDF: `ML_HW2_ID1_ID2.pdf`

The assignment should be submitted in pairs, with only one submission per pair. The submitted notebook and HTML must include all sections and all visible outputs. The presentation should be submitted as a PDF.

The file names should use the form `ML_HW2_ID1_ID2` , where `ID1` and `ID2` are the student IDs.

Assignments submitted late will receive a penalty of 3 points for each day, up to one week. Later submissions will not be accepted.

Grading and Academic Guidelines

The assignment will be graded according to correctness, clarity, efficiency of implementation, and elegance of implementation.

Self-learning is an important part of the course. Treat all sources of information carefully and critically. Use of LLMs is allowed, but you must briefly state where and how you used them.

You may consult with other students in the course, but each pair must write and submit its own work. It is reasonable that not all results and algorithms will be identical.

Dataset limitation: `can_answer` is derived from how the original benchmark examples were constructed, not from new human annotations made specifically for this course. Therefore, the label is useful for this assignment, but it should not be treated as perfect real-world ground truth about whether an AI agent should call a tool. Some models may learn patterns from the benchmark construction and keyword-based features rather than fully general tool-use reasoning.

Major penalties:

- using `can_answer` as an input feature,
- using raw IDs as predictive features,
- tuning models on the test set,
- submitting a notebook that does not run,
- showing plots without explaining what they mean.

Questions and Reception Hours

Please post professional questions about the assignment on the Moodle forum after reading previous posts. Professional questions sent by email will not be answered.

If you want to schedule a reception hour with one of the instructors, send your questions by email in advance. For personal questions, justified extension requests, or similar issues, email the instructor.

Good luck.